

RippleLab Day 03 Progress Report

Day: 03 of 30

Date: 2026-08-13

Status: Complete

Branch: codex/day-03-authentication

Implementation commit: 06e34dd

CI: Full local quality gate passed

1. Objective

Add a secure, understandable account entry flow backed by Supabase SSR, protect the private dashboard at both the request and server-render boundaries, and cover sign-up, sign-in, sign-out and password recovery without committing credentials.

2. Acceptance Criteria

- [x] Anonymous users are redirected away from `/dashboard` to sign in.
- [x] Sign-up, sign-in and sign-out flows are implemented.
- [x] Password recovery and new-password flows are implemented.
- [x] Auth forms validate on the server and show understandable field/provider errors.
- [x] Supabase sessions use the current SSR cookie pattern and refresh proxy.
- [x] A test account completes the full lifecycle in automated browser coverage.
- [x] Authentication pages work on desktop and mobile with no critical or serious axe violations.
- [x] Local setup documents the required provider URL, key and redirect configuration.

3. Completed Work

RippleLab now has a public landing page and a private dashboard boundary. Visitors can create an account, sign in, request a recovery email, choose a new password and sign out. Hosted accounts use Supabase's PKCE-compatible email confirmation and recovery callback route.

The dashboard is protected twice: Next.js Proxy performs an early request redirect, and the dashboard server layout verifies the user again before rendering private content. Authenticated users see their account email and a sign-out control in the application shell.

When Supabase is not configured, public account forms remain viewable and return a clear local-setup message instead of crashing. No service-role key is used by the browser flow.

4. Technical Implementation

- Added `@supabase/ssr` and `@supabase/supabase-js` clients for browser, Server Components, Server Actions and Proxy.
- Implemented only the current `getAll` and `setAll` cookie adapter pattern.

- Added request-time /dashboard rendering and server authorization.
- Added Zod schemas for email, password, matching confirmation and recovery inputs.
- Added React 19 useActionState forms with pending, field-error and provider-error states.
- Added a /auth/confirm callback accepting Supabase authorization codes and token-hash email templates for PKCE confirmation and recovery.
- Added a non-production-only test provider. Its cookie state exists solely to test browser behavior without embedding real Supabase credentials and is hard-disabled when NODE_ENV is production.
- Expanded Playwright to run serially for stable development-server behavior across desktop and mobile Chromium.

5. Calculations, Assumptions and Evidence

No economic calculations changed. Authentication implementation follows current Supabase SSR documentation reviewed on 2026-08-13 and the bundled Next.js 16.3 authentication, Proxy, Server Actions, cookies, redirect and request-time rendering guides.

Passwords require at least 12 characters, at least one letter and at least one number at the application boundary. Supabase project-level password and leaked-password policies remain authoritative in hosted environments.

6. Verification Evidence

Check	Command or method	Result
Shared contract	<code>pnpm check:contracts</code>	Passed
Web lint	ESLint	Passed
Web type-check	TypeScript	Passed
Production build	Next.js 16.3 webpack build	Passed; auth Proxy and dynamic dashboard emitted
Browser suite	Playwright desktop/mobile Chromium	Passed; 12 tests, 2 intentional device skips
Auth lifecycle	Anonymous redirect, sign-up, sign-out, failed sign-in, recovery, password update, new sign-in	Passed
Accessibility	axe-core on public, gallery, sign-in, sign-up and recovery routes	Passed with no serious or critical violations
API checks	Ruff and pytest	Passed; 2 tests
Secret scan	<code>pnpm check:secrets</code>	Passed
Full quality gate	<code>pnpm check</code>	Passed
Visual review	Sign-up at desktop and 390 x 844, plus protected redirect	Passed
Git whitespace	<code>git diff --check</code>	Passed

7. Important Files Changed

- `apps/web/src/proxy.ts` - Supabase session refresh and optimistic route redirect.
- `apps/web/src/lib/supabase` - environment-safe browser and server clients.
- `apps/web/src/lib/auth` - validation, trusted session lookup and isolated test provider.

- `apps/web/src/app/auth/actions.ts` - server-side auth mutations and error mapping.
- `apps/web/src/app/auth` - sign-in, sign-up, recovery, confirmation and update-password routes.
- `apps/web/src/app/dashboard` - request-time protected dashboard layout and page.
- `apps/web/src/components/auth` - reusable account layout and action-state form.
- `apps/web/tests/auth.spec.ts` - full browser lifecycle.
- `.env.example` and `README.md` - safe Supabase setup and redirect guidance.

8. Visual Proof

The sign-up experience was visually inspected at the default desktop viewport and 390 x 844 mobile. The two-panel desktop composition becomes a readable stacked mobile flow with visible labels, password guidance and disclaimer. A direct anonymous visit to `/dashboard` was also verified to render the sign-in screen through a redirect.

9. Security, Privacy and Accessibility

- User identity is verified through `auth.getUser()` rather than trusting raw session data.
- Protected data has a server-render authorization check in addition to Proxy.
- Redirect destinations accept only same-origin paths beginning with one slash.
- Provider errors do not expose account existence during recovery.
- Passwords are never logged, persisted in source or exposed in error messages.
- Supabase service-role credentials are not required and remain server-only placeholders.
- Auth test mode is unavailable in production.
- Forms expose associated labels, descriptions, invalid states and live status messages.

10. Decisions

- Use Supabase's publishable-key terminology while accepting the legacy anon-key environment variable during migration.
- Use Server Actions for credential mutations and expected-error return values.
- Keep Proxy as an optimistic first line, with server verification as the authorization boundary.
- Use a test-only provider adapter rather than committing or borrowing real credentials for CI.
- Serialize browser tests to avoid intermittent Next.js development compilation contention.

11. Risks and Blockers

- A real hosted Supabase project has not been supplied, so live SMTP delivery and hosted-user lifecycle could not be exercised. Production wiring requires project URL, publishable key, Site URL and allowed callback configuration.
- Supabase's default email sender is rate-limited and not intended for production; custom SMTP is required before launch.
- Row-level security begins with the profile schema on Day 4 and remains mandatory before storing financial data.

No code blocker remains for Day 3.

12. GitHub Publication

- Implementation commit message: `feat(auth): add Supabase authentication flow`
- Implementation commit SHA: `06e34dd`
- Remote branch: `codex/day-03-authentication`
- Pull request: Not requested
- CI status: Local quality gate passed; remote workflow status pending push

13. Next Day

Day 4 will define the personal financial profile schema and migrations, row-level security, multi-step profile capture, rupee/percentage validation, consent copy and create/edit/reload behavior for the authenticated user.

Sign-off

Prepared by: Codex

Evidence reviewed: Yes

Ready to continue: Yes, with a Supabase project required for live hosted-auth verification